

Spis treści

Cel i zakres projektu	2
Wykorzystane technologie	2
Opis funkcjonalności z dokumentacją wizualną	3
1. Interfejs graficzny, zarządzanie plikami i interaktywny widok roboczy	3
2. Wykrywanie krawędzi	4
a) Detektor Canny'ego	4
b) Filtr Sobela	6
c) Operator Laplace'a	7
3. Progowanie obrazu	8
a) Progowanie binarne	8
b) Metoda Otsu	9
c) Progowanie adaptacyjne	10
4. Przetwarzanie wsadowe	11
5. Analiza obrazu – dynamiczny histogram RGB	13
6. Klasyfikacja obiektów – Faster R-CNN	14
7. Detekcja ras kotów – YOLOv8	15
8. Detekcja pól i symulacja szkieletowa	16
9. Efekt „smażenia obrazu na głębokim oleju”	17
10. Podmiana twarzy i sortowanie pikseli według jasności	18
Faza I – dopasowanie geometryczne i podmiana twarzy:	19
Faza II – transport pikseli według rangi jasności:	20
11. „Game of Life” Conwaya	21
Kod źródłowy projektu	22

Cel i zakres projektu

Celem projektu jest stworzenie zaawansowanej, interaktywnej aplikacji desktopowej przeznaczonej do przetwarzania i interpretacji obrazów cyfrowych. Sprawozdanie dokumentuje pełną realizację projektu obejmującego interfejs graficzny, zestaw algorytmów filtracji i binaryzacji lokalnej, moduł dynamicznej analizy statystycznej w postaci histogramów RGB, a także hybrydowy system sztucznej inteligencji do detekcji obiektów i estymacji pól człowieka. W ramach projektu zaimplementowano ponadto system przetwarzania wsadowego, autorski efekt wizualny 'smażenia obrazu' oraz automat komórkowy Conwaya zintegrowany z wynikami detekcji krawędzi.

Wykorzystane technologie

Język programowania: Python 3.13

Interfejs graficzny: PyQt6 – framework aplikacyjny do budowy struktury okien i obsługi zdarzeń; pywinstyles – biblioteka do natywnej implementacji półprzezroczystego stylu Frutiger Aero

Przetwarzanie obrazu i matematyka macierzowa: OpenCV – silnik przetwarzania graficznego odpowiedzialny za transformacje przestrzeni, filtracje, wykrywanie krawędzi binaryzacje i obsługę wejścia/wyjścia; NumPy – biblioteka do niskopoziomowej optymalizacji operacji na macierzach pikseli

Wizualizacja danych statystycznych: Matplotlib – narzędzie zintegrowane z Qt do dynamicznego generowania wykresów histogramu RGB

Sztuczna inteligencja i Deep Learning: PyTorch oraz Torchvision – framework do obsługi modelu Faster R-CNN z architekturą ResNet-50; Ultralytics YOLOv8 – platforma detekcyjna do ekstrakcji punktów kluczowych szkieletu i detekcji ras kotów

Obsługa multimediów: PyQt6.QtMultimedia – moduł do asynchronicznego zarządzania strumieniem audio

Przygotowanie danych i trening modeli: Google Colab, platforma Roboflow

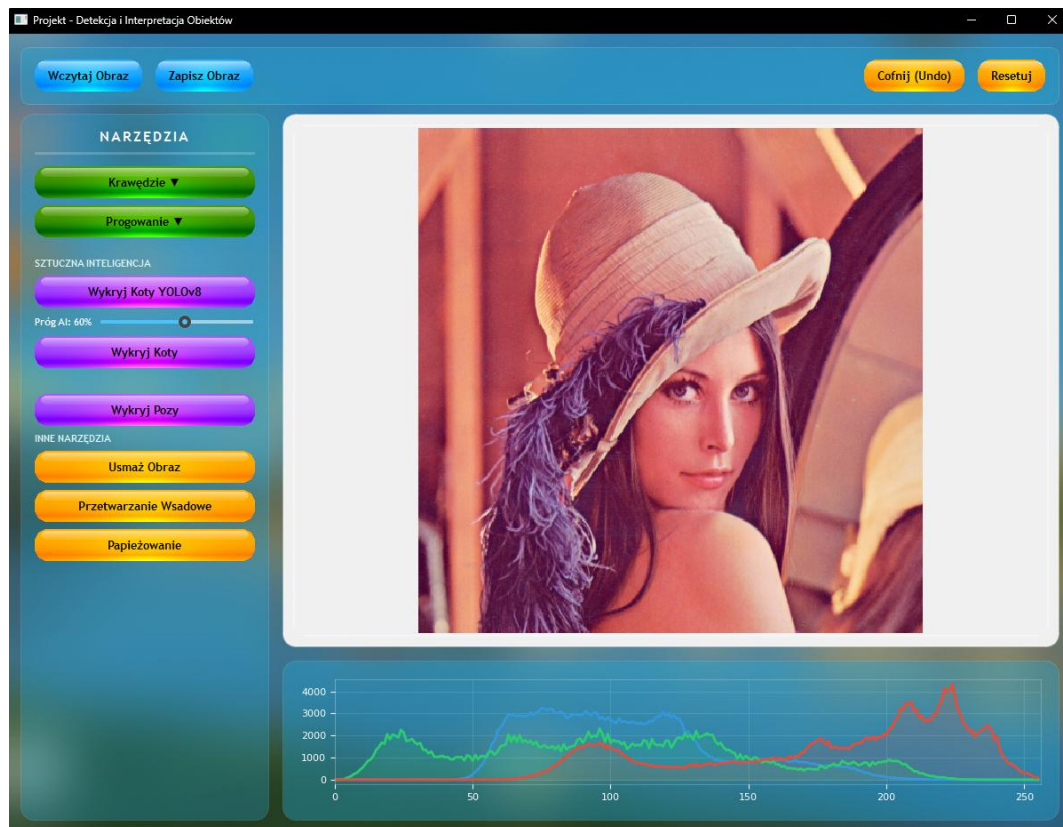
Opis funkcjonalności z dokumentacją wizualną

1. Interfejs graficzny, zarządzanie plikami i interaktywny widok roboczy

Interfejs aplikacji jest podzielony na 4 części. Pasek boczny zawiera wszystkie narzędzia do przetwarzania i detekcji obiektów, podgląd obrazu znajduje się w oknie po prawej stronie, na dole aplikacji znajduje się histogram obrazu odświeżający się z każdą zmianą na obrazie, a na górnym pasku są przyciski odpowiedzialne za wczytanie, zapis i cofanie zmian przeprowadzonych na obrazie. Zaimplementowano bezpieczne wczytywanie i zapis obrazów z obsługą polskich znaków w ścieżkach systemowych, cofanie operacji przez stos historii stanów oraz swobodne przybliżanie i przesuwanie obszaru roboczego. Wczytywanie pliku metodą przeciągnij-i-upuść skraca czas potrzebny do załadowania materiału. Przytrzymanie prawego przycisku myszy przełącza podgląd na oryginalny obraz bez resetowania aktualnego poziomu przybliżenia.

Silnik wyświetlania obrazu oparty na klasach QGraphicsView i QGraphicsScene. Bezpieczne wczytywanie plików ze znakami diakrytycznymi realizowane jest przez buforowanie danych funkcją `numpy.fromfile` i dekodowanie zawartości w `cv2.imdecode`, z pominięciem systemowej obsługi ścieżek.

```
file_bytes = np.fromfile(file_path, dtype=np.uint8)
    # wczytanie pliku jako surowe bajty - ominięcie problemu diakrytyków w
    # ścieżce
self.original_image = cv2.imdecode(file_bytes, cv2.IMREAD_COLOR)
    # dekodowanie bufora bajtów do macierzy BGR
self.processed_image = self.original_image.copy()
    # kopia robocza niezależna od oryginału
self.history = []
    # zerowanie stosu historii przy nowym obrazie
```



Rys. 1. Interfejs główny aplikacji

2. Wykrywanie krawędzi

a) Detektor Canny'ego

Detektor Canny'ego realizuje wielostopniowy potok przetwarzania: wygładzanie filtrem Gaussa, obliczanie gradientów metodą Sobela, nielokalne tłumienie maksimów i dwuprogowe śledzenie krawędzi przez histerezę. Dolny próg wynosi 100, górny 200 – wartości dobrane empirycznie w celu wyodrębnienia cienkich, ciągłych konturów.

```
gray = cv2.cvtColor(self.processed_image, cv2.COLOR_BGR2GRAY)
# redukcja do jednokanałowego obrazu intensywności
edges = cv2.Canny(gray, 100, 200)
# potok: Gauss → gradient Sobela → NMS → histereza z T_low=100, T_high=200
self.processed_image = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
# rozszerzenie do 3 kanałów BGR wymaganego przez moduł wyświetlania
```

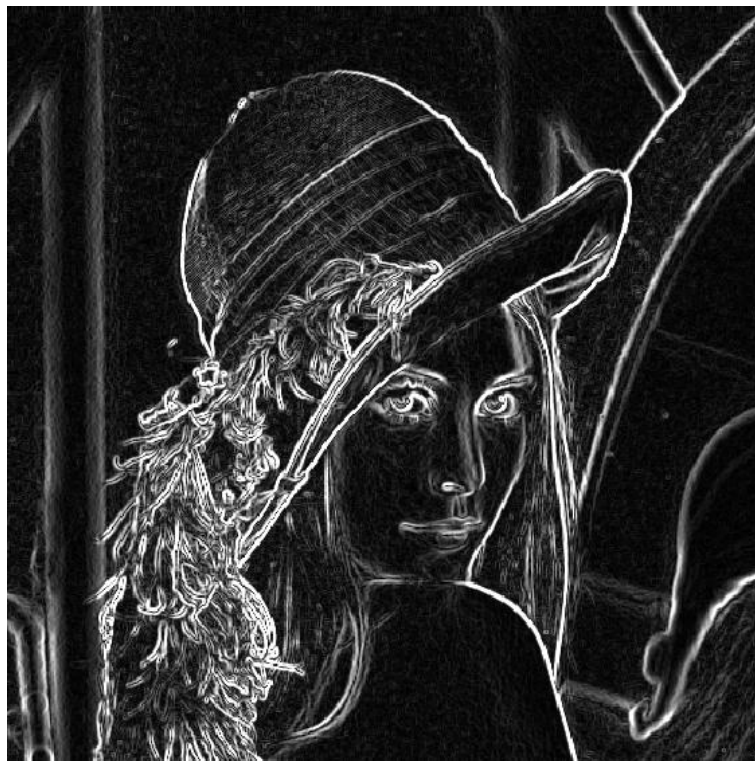


Rys. 2. Wynik działania detektora Canny'ego

b) Filtr Sobela

Filtr Sobela oblicza pochodne cząstkowe pierwszego rzędu obrazu w kierunkach poziomym i pionowym. Połączenie obu składowych przez moduł wektora gradientu daje obraz intensywności krawędzi, odzwierciedlający kierunek i siłę zmian jasności.

```
gray = cv2.cvtColor(self.processed_image, cv2.COLOR_BGR2GRAY)
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
    # pochodna cząstkowa dI/dx - jądrowo 3x3 w kierunku poziomym
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
    # pochodna cząstkowa dI/dy - jądrowo 3x3 w kierunku pionowym
sobel_combined = cv2.magnitude(sobelx, sobely)
    # moduł gradientu: |grad I| = sqrt(Gx^2 + Gy^2)
sobel_8u = cv2.convertScaleAbs(sobel_combined)
    # skalowanie wartości float64 do zakresu uint8
```



Rys. 3. Wynik działania filtru Sobela

c) Operator Laplace'a

Operator Laplace'a oblicza drugą pochodną przestrzenną obrazu i wykrywa obszary gwałtownych zmian jasności we wszystkich kierunkach jednocześnie. Ze względu na wysoką czułość na szumy, przed zastosowaniem operatora obraz wygładzany jest filtrem Gaussa.

```
gray = cv2.cvtColor(self.processed_image, cv2.COLOR_BGR2GRAY)
blurred = cv2.GaussianBlur(gray, (3, 3), 0)
    # wygładzanie Gaussa tłumiące szumy WCZ przed różniczkowaniem
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)
    # operator Laplace'a:  $Lap(I) = d^2I/dx^2 + d^2I/dy^2$ 
laplacian_8u = cv2.convertScaleAbs(laplacian)
    # normalizacja przez  $|x|$  do zakresu uint8
```



Rys. 4. Wynik działania operatora Laplace'a

3. Progowanie obrazu

a) Progowanie binarne

Progowanie binarne dzieli obraz na dwa regiony na podstawie jednego stałego progu jasności T . Każdy piksel o wartości powyżej T zostaje ustawiony na 255, poniżej na 0. Metoda ta nie uwzględnia lokalnych warunków oświetlenia, przez co jest wrażliwa na niejednorodność źródła światła.

```
gray = cv2.cvtColor(self.processed_image, cv2.COLOR_BGR2GRAY)
_, thresh_img = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
# binaryzacja:  $f(x,y) = 255$  gdy  $I(x,y) > T=127$ , inaczej 0
self.processed_image = cv2.cvtColor(thresh_img, cv2.COLOR_GRAY2BGR)
```



Rys. 5. Wynik progowania binarnego ze stałym progiem $T = 127$

b) Metoda Otsu

Metoda Otsu automatycznie wyznacza optymalny próg binaryzacji przez minimalizację ważonej wariancji wewnątrzklasowej dwóch grup pikseli. Analiza histogramu zastępuje konieczność ręcznego ustawiania progu.

```
gray = cv2.cvtColor(self.processed_image, cv2.COLOR_BGR2GRAY)
_, thresh_img = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
# wyznaczenie progu  $T^*$  minimalizującego  $sw^2(T) = w1*s1^2 + w2*s2^2$ 
self.processed_image = cv2.cvtColor(thresh_img, cv2.COLOR_GRAY2BGR)
```



Rys. 6. Wynik progowania metodą Otsu

c) Progowanie adaptacyjne

W progowaniu adaptacyjnym próg wyznaczany jest lokalnie dla każdego fragmentu obrazu jako ważona średnia Gaussa wartości pikseli w oknie sąsiedztwa. Metoda zachowuje szczegóły obiektów w warunkach nierównomiernego oświetlenia.

```
gray = cv2.cvtColor(self.processed_image, cv2.COLOR_BGR2GRAY)
thresh_img = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    #  $T(x,y)$  = ważona średnia Gaussa sąsiedztwa 11x11
    cv2.THRESH_BINARY, 11, 2
    # rozmiar bloku lokalnego = 11x11, stała  $C=2$  odejmowana od średniej
)
self.processed_image = cv2.cvtColor(thresh_img, cv2.COLOR_GRAY2BGR)
```



Rys. 7. Wynik progowania adaptacyjnego z lokalnym wyznaczaniem progu

4. Przetwarzanie wsadowe

Moduł przetwarzania wsadowego umożliwia jednoczesną zmianę rozdzielczości i nałożenie znaku wodnego na wszystkie obrazy z wybranego folderu. Wyniki zapisywane są do wskazanego katalogu docelowego. Zaimplementowano pełną obsługę przezroczystości PNG – znak wodny wczytywany jest flagą `IMREAD_UNCHANGED`, co zachowuje czwarty kanał Alfa i umożliwia nakładanie logo z precyzyjną maską przezroczystości wyznaczaną piksel-po-pikselu z pliku źródłowego. Znak wodny skalowany jest proporcjonalnie do 20 procent szerokości każdego obrazu docelowego i wstawiany w prawym dolnym rogu z maksymalnym kryciem na poziomie 50 procent. Dla plików bez kanału alfa stosowane jest awaryjne blendowanie addytywne.

```
wm_bytes = np.fromfile(watermark_path, dtype=np.uint8)
watermark = cv2.imdecode(wm_bytes, cv2.IMREAD_UNCHANGED)
    # IMREAD_UNCHANGED: wczytanie 4-kanałowej struktury BGRA z zachowaniem
    # kanału alfa

img = cv2.resize(img, target_size, interpolation=cv2.INTER_AREA)
    # INTER_AREA: uśrednianie pikseli obszaru źródłowego redukuje aliasing przy
    # zmniejszaniu

wm_w = int(w_img * 0.2)
    # szerokość znaku wodnego = 20% szerokości obrazu docelowego
scale = wm_w / watermark.shape[1]
wm_h = int(watermark.shape[0] * scale)
    # proporcjonalna wysokość zachowująca proporcje znaku wodnego
wm_resized = cv2.resize(watermark, (wm_w, wm_h), interpolation=cv2.INTER_AREA)
    # skalowanie 4-kanałowego znaku wodnego z zachowaniem kanału alfa

if wm_resized.shape[2] == 4:
    wm_rgb = wm_resized[:, :, :3]
        # wyodrębnienie kanałów BGR koloru znaku wodnego
    alpha_mask = wm_resized[:, :, 3] / 255.0
        # normalizacja kanału alfa do zakresu 0.0-1.0 jako maska pikseli
    alpha_mask_3ch = cv2.merge([alpha_mask, alpha_mask, alpha_mask])
        # duplikacja maski na 3 kanały dla operacji macierzowej

    logo_opacity = 0.5
    alpha_mask_3ch = alpha_mask_3ch * logo_opacity
        # ograniczenie widoczności znaku wodnego do maksymalnie 50%

    blended = roi * (1.0 - alpha_mask_3ch) + wm_rgb * alpha_mask_3ch
        # blending piksel-po-pikselu: wynik = tło*(1-alfa) + logo*alfa
    img[y1:y2, x1:x2] = blended.astype(np.uint8)
else:
    blended = cv2.addWeighted(roi, 0.7, wm_resized, 0.3, 0)
        # fallback dla plików bez kanału alfa – jednolite blendowanie addytywne
    img[y1:y2, x1:x2] = blended
```

Przetwarzanie Wsadowe

×

1. Folder źródłowy:

obrazy

Wybierz folder

2. Znak wodny (opcjonalnie):

red-among-us-png-842x1024.png

Wybierz plik

3. Zmiana rozdzielczości (opcjonalnie):

100

200

ROZPOCZNIJ PRZETWARZANIE

Rys. 8. Panel konfiguracji przetwarzania wsadowego

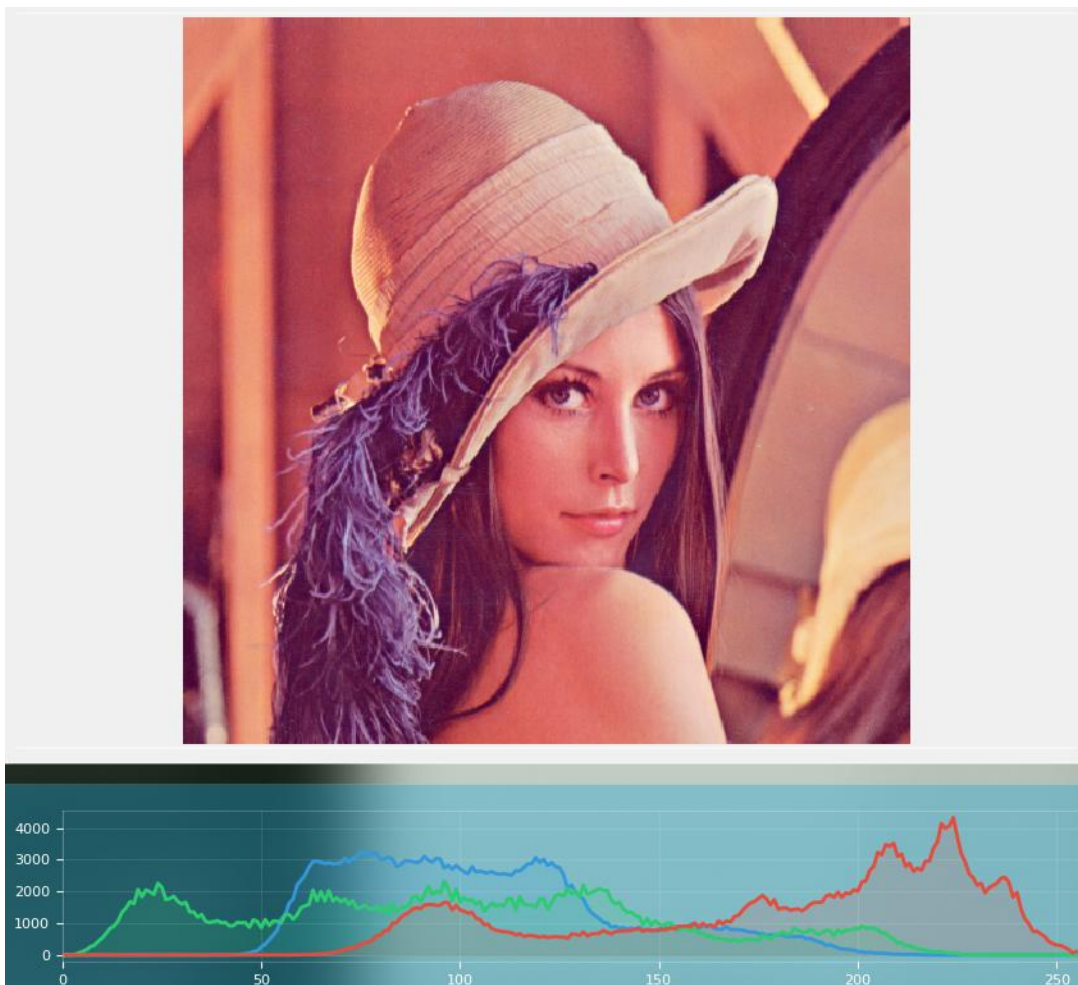


Rys. 9. Obraz wynikowy z prawidłowo nałożonym znakiem wodnym z maską Alfa

5. Analiza obrazu – dynamiczny histogram RGB

Histogram RGB generowany i aktualizowany jest automatycznie po każdej operacji na obrazie. Wykres przedstawia rozkład jasności oddzielnie dla kanałów niebieskiego, zielonego i czerwonego przestrzeni BGR. Renderowanie wykresu wektorowego wewnątrz GUI realizowane jest przez bibliotekę Matplotlib połączoną z PyQt6 przez klasę FigureCanvasQTagg.

```
hist_data = []
for i in range(3):
    hist = cv2.calcHist(
        [self.processed_image], [i],
        # i-ty kanał: 0=B, 1=G, 2=R
        None,
        [256],
        # 256 przedziałów = pełny zakres intensywności
        [0, 256]
        # zakres wartości piksela
    )
    hist_data.append(hist)
```



Rys. 10. Wykres rozkładu kanałów kolorystycznych RGB

6. Klasyfikacja obiektów – Faster R-CNN

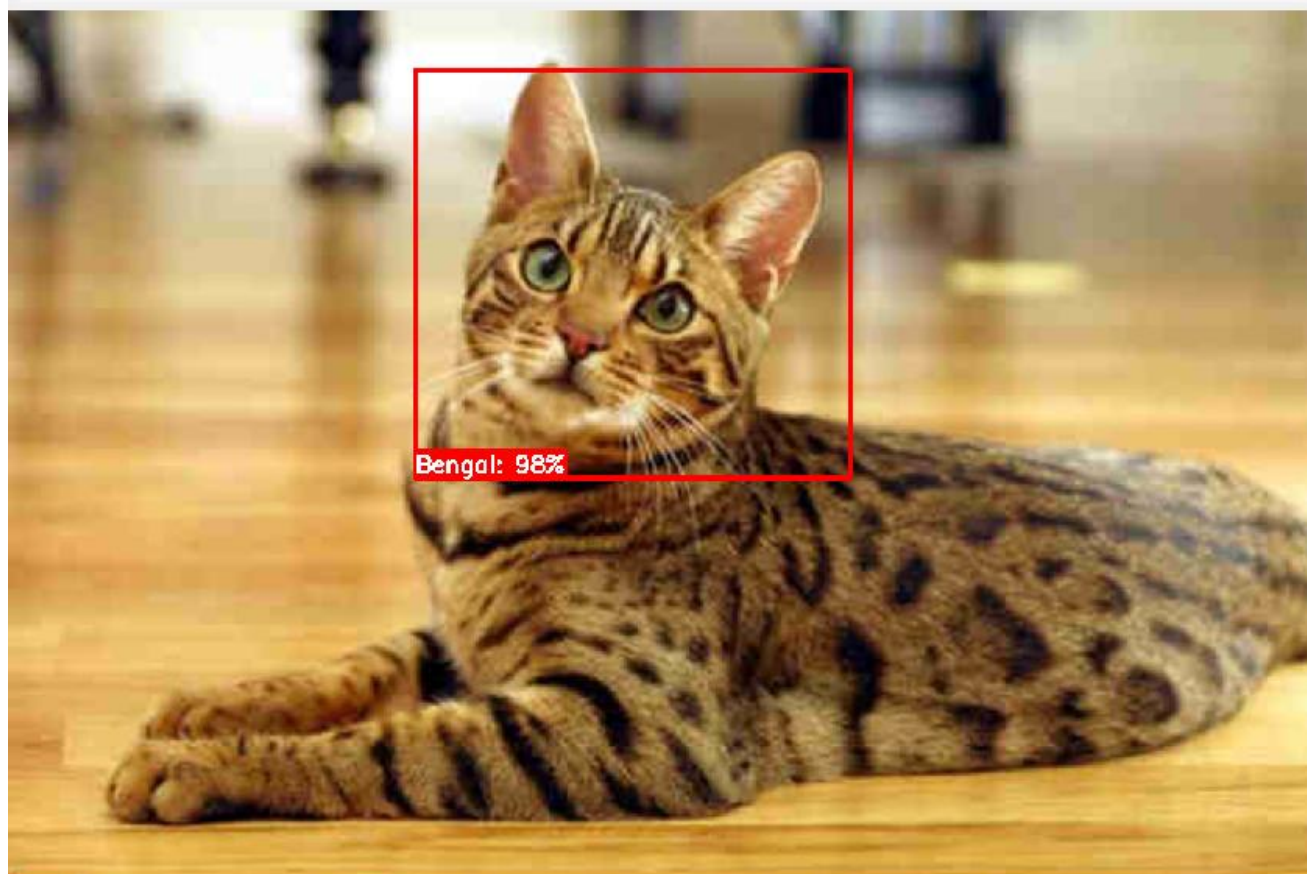
Aplikacja wykorzystuje model głębokiej sieci neuronowej Faster R-CNN z kręgosłupem ResNet-50. Model dotrenowany został do rozpoznawania pięciu ras kotów: Bengal, Persian, Ragdoll, Russian Blue i Siamese. Na obrazie rysowane są czerwone ramki ograniczające z podpisami zawierającymi nazwę rasy i wartość pewności predykcji. Próg akceptacji wyników ustawiany jest dynamicznie przez użytkownika suwakiem interfejsu w zakresie od 10 do 100 procent.

```
img_tensor = TF.to_tensor(img_rgb).unsqueeze(0).to(self.device)
    # normalizacja pikseli do [0.0, 1.0]; dodanie wymiaru batch → [1, 3, H, W]

with torch.no_grad():
    predictions = self.nn_model(img_tensor)[0]
    # inferencja bez gradientów - tryb ewaluacji; zwraca słownik z
    # boxes/labels/scores

boxes = predictions['boxes'].cpu().numpy()
    # ramki detekcji w formacie [x_min, y_min, x_max, y_max]
scores = predictions['scores'].cpu().numpy()
    # pewności predykcji w zakresie [0.0, 1.0]

for box, label, score in zip(boxes, labels, scores):
    if score >= threshold:
        # odrzucenie predykcji poniżej progu ustawionego przez użytkownika
        cv2.rectangle(self.processed_image,
            (x_min, y_min), (x_max, y_max), (0, 0, 255), 3)
        # czerwona ramka detekcji BGR
        cv2.putText(self.processed_image, text, (x_min, y_max - 5),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)
        # podpis z nazwą rasy i wartością pewności
```

Rys. 11. Wynik detekcji ras kotów modelem Faster R-CNN

7. Detekcja ras kotów – YOLOv8

Aplikacja zawiera drugi, niezależny moduł detekcji ras kotów oparty na modelu YOLOv8n wytrenowanym na zbiorze danych przygotowanym na platformie Roboflow. Ramki detekcji rysowane są kolorem zielonym, co pozwala odróżnić wyniki od modelu Faster R-CNN działającego równolegle. Architektura YOLOv8n przetwarza obraz jednym przejściem sieci, co przekłada się na niższe czasy inferencji.

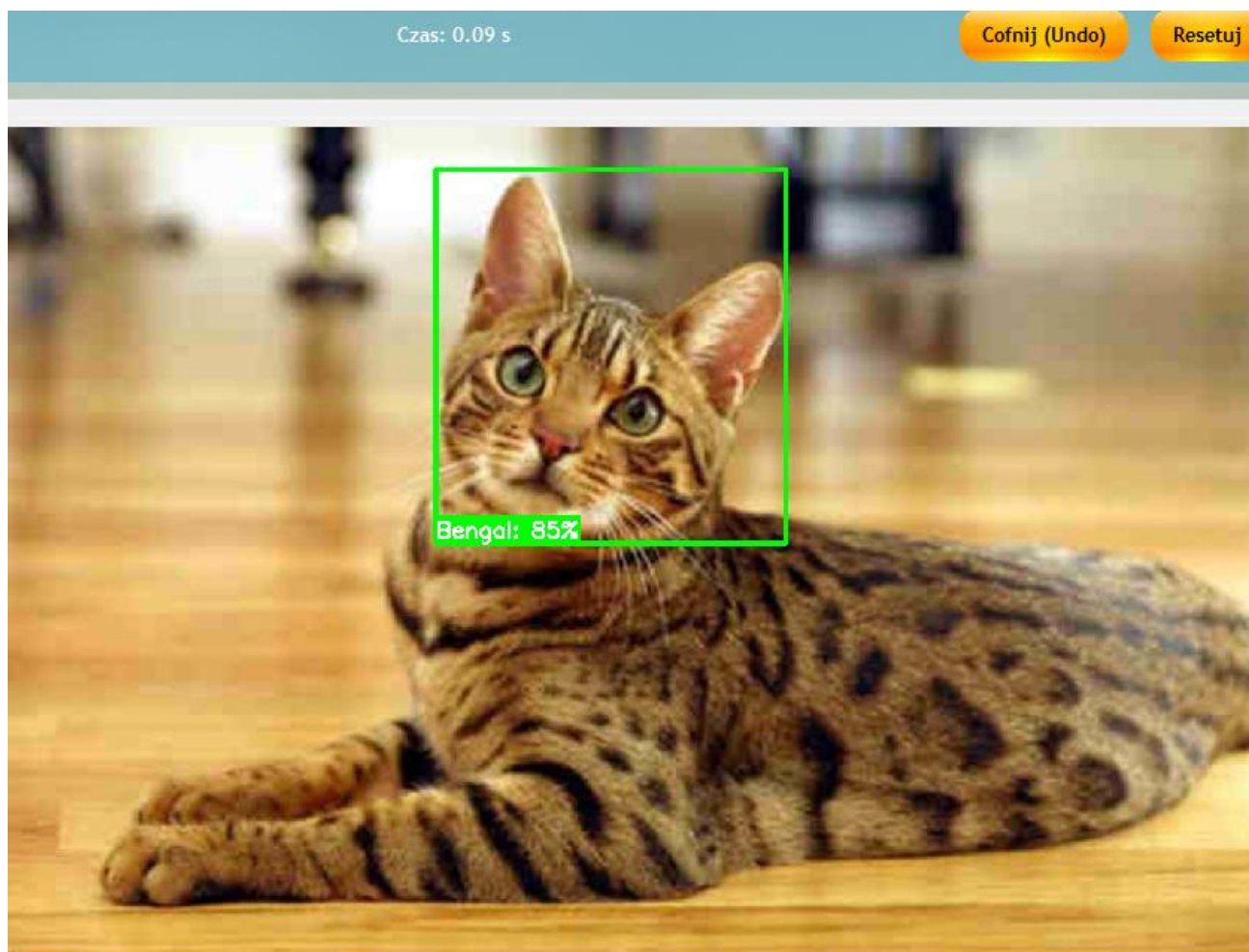
```
self.yolo_cat_model = YOLO('best_n.pt')
    # załadowanie niestandardowego modelu YOLOv8n wytrenowanego na rasach kotów

results = self.yolo_cat_model(self.processed_image, verbose=False)
    # inferencja jednym przejściem sieci - single-pass detection
boxes = results[0].boxes
    # obiekty detekcji z polami conf, cls, xyxy

for box in boxes:
    conf = float(box.conf[0])
    # pewność predykcji danej ramki
    if conf >= threshold:
        # filtrowanie wyników poniżej progu akceptacji
        cls_id = int(box.cls[0])
        # identyfikator wykrytej klasy
        x_min, y_min, x_max, y_max = map(int, box.xyxy[0])
```



```
# współrzędne ramki w formacie xyxy
cv2.rectangle(self.processed_image,
              (x_min, y_min), (x_max, y_max), (0, 255, 0), 3)
# zielona ramka detekcji YOLOv8 BGR
```



Rys. 12. Wynik detekcji ras kotów modelem YOLOv8

8. Detekcja póz i symulacja szkieletowa

Moduł wykrywania póz wykorzystuje model YOLOv8n-Pose do detekcji sylwetek ludzkich i naniesienia na nie szkieletu złożonego z 17 punktów kluczowych standardu COCO. Punkty kluczowe obejmują oczy, nos, tułów i kończyny. Wbudowana metoda plot() generuje wizualizację szkieletu bezpośrednio na macierzy obrazu jako warstwę wynikową, zapewniając wysoką wydajność analizy.

```
self.pose_model = YOLO('yolov8n-pose.pt')
    # model YOLOv8n-Pose - wariant Nano z regresją 17 punktów kluczowych COCO

results = self.pose_model(self.processed_image, verbose=False)
    # inferencja: detekcja sylwetek + regresja keypoints jednym przejściem sieci

annotated_image = results[0].plot()
    # nałożenie wektorowej wizualizacji szkieletu na macierz BGR

self.processed_image = annotated_image
```

| # podmiana obrazu roboczego na wersję z naniesionymi szkieletami



Rys. 13. Wynik detekcji póż z wizualizacją 17 punktów kluczowych szkieletu

9. Efekt „smażenia obrazu na głębokim oleju”

Autorski efekt wizualny ekstremalnie przesterowuje obraz przez potok kolejnych operacji: agresywne wyostżanie konwolucją, podbicie saturacji w przestrzeni HSV, zwiększenie kontrastu i jasności oraz dodanie szumu addytywnego. Nakładanie efektu animowane jest przez 60 klatek z interpolacją alfa, a procesowi towarzyszy trzysekundowy efekt dźwiękowy skwierczenia. Animacja realizowana jest przez interpolację liniową między obrazem oryginalnym i usmażonym przy użyciu `cv2.addWeighted`.

```
kernel_sharpen = np.array([[ -1, -1, -1],
                           [ -1,  9, -1],
                           [ -1, -1, -1]])
# jądno wyostżania: wzmacnia różnicę piksela od średniej sąsiedztwa
img = cv2.filter2D(img, -1, kernel_sharpen)
# splot 2D: piksel zastąpiony sumą ważoną w oknie 3x3

hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
h, s, v = cv2.split(hsv)
# rozdzielenie kanałów Hue / Saturation / Value
```

```

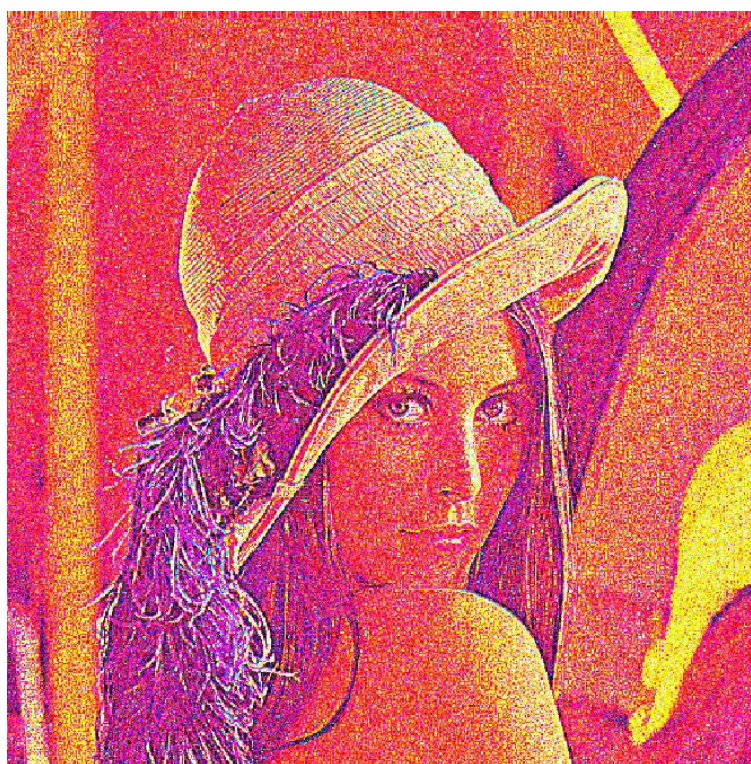
s = cv2.multiply(s, 4.0)
    # saturacja S mnożona przez 4 - czterokrotne nasycenie barw
s = np.clip(s, 0, 255).astype(np.uint8)
    # obcięcie do zakresu uint8 po przekroczeniu 255
img = cv2.cvtColor(cv2.merge([h, s, v]), cv2.COLOR_HSV2BGR)

img = cv2.convertScaleAbs(img, alpha=2.0, beta=30)
    # kontrast:  $I' = 2.0 * I + 30$  - liniowe przekształcenie jasności

noise = np.random.randint(-60, 60, img.shape, dtype='int16')
    # szum addytywny z rozkładu jednostajnego  $U[-60, 60]$ 
img = np.clip(img.astype('int16') + noise, 0, 255).astype('uint8')
    # dodanie szumu z zabezpieczeniem zakresu przez int16

blended = cv2.addWeighted(orig_img, alpha, fried_img, 1.0 - alpha, 0)
    # animacja: interpolacja liniowa alfa przez 60 klatek

```



Rys. 14. Wynik efektu smażenia

10. Podmiana twarzy i sortowanie pikseli według jasności

Funkcja Algorytm realizuje przetwarzanie w dwóch niezależnych fazach. W pierwszej fazie model YOLOv8-Pose wykrywa na obrazie kluczowe punkty anatomiczne twarzy – nos i obie gałki oczne – a następnie naciąga na nie twarz docelową przez obliczenie macierzy przekształcenia geometrycznego. Macierz wyznaczana jest z trzech par odpowiadających sobie punktów i opisuje unikalne przesunięcie, obrót i skalowanie twarzy docelowej tak, aby jej punkty charakterystyczne pokryły się dokładnie z punktami wykrytymi na zdjęciu. Nakładka obracana jest o 5 stopni CCW wokół punktu nosa.

W przypadku braku wykrytej twarzy algorytm przechodzi do drugiej fazy – sortowania pikseli według rangi jasności. Metodą Coarse-to-Fine obraz dzielony jest na bloki malejącej wielkości, a piksele bloku źródłowego przypisywane są do pozycji docelowych zgodnie z ich miejscem w rankingu luminancji. Kluczowym elementem jest maska aktywnych pikseli, która wyklucza z sortowania obszary białego tła i ogranicza operację wyłącznie do pikseli należących do obszaru twarzy docelowej.

Faza I – dopasowanie geometryczne i podmiana twarzy:

```
pythonR = cv2.getRotationMatrix2D((float(nose[0]), float(nose[1])), angle, 1.0)
    # macierz obrotu 2D wokół punktu nosa; angle=5 oznacza obrót o 5 stopni CCW

pts_ones = np.hstack([pts_eyes, np.ones((2, 1))])
rotated_eyes = R.dot(pts_ones.T).T
    # obrót punktów oczu przez mnożenie macierzowe:  $p' = R * p$ 

M = cv2.getAffineTransform(pts_jp2, pts_img)
    # wyznaczenie macierzy 2x3 przekształcenia geometrycznego z 3 par punktów
    korespondencyjnych
    # macierz opisuje obrót, skalowanie i przesunięcie naciągające twarz
    docelową na wykrytą

warped_jp2 = cv2.warpAffine(tgt_face, M, (w_img, h_img), borderValue=(255, 255,
255))
    # zastosowanie macierzy: naciągnięcie twarzy docelowej na punkty kluczowe
    twarzy wykrytej

jp2_mask = cv2.GaussianBlur(jp2_mask, (21, 21), 0)
    # rozmycie krawędzi eliptycznej maski twarzy filtrem Gaussa

mask_3ch = cv2.cvtColor(warped_mask, cv2.COLOR_GRAY2BGR) / 255.0
    # normalizacja maski do zakresu 0.0-1.0 jako wagi dla alfa-blendingu

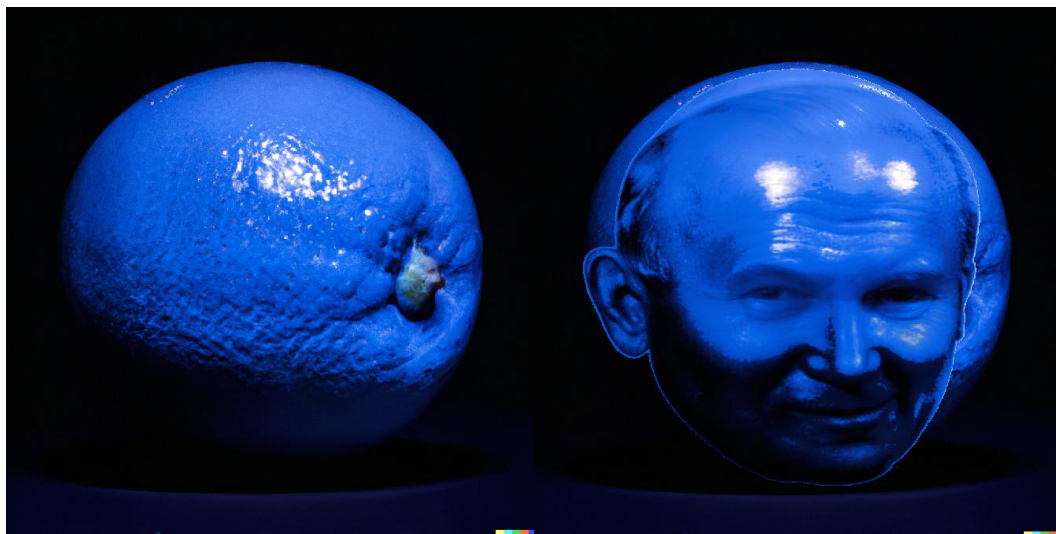
T_canvas = (T_canvas * (1.0 - mask_3ch) + warped_jp2 *
mask_3ch).astype(np.uint8)
    # kompozytowanie: wynik = tło*(1-alfa) + nakładka*alfa
```



Rys. 15. Wynik podmiany twarzy z dopasowaniem geometrycznym do punktów kluczowych

Faza II – transport pikseli według rangi jasności:

```
pythonT_luma = 0.114*T_canvas[:, :, 0] + 0.587*T_canvas[:, :, 1] +  
0.299*T_canvas[:, :, 2]  
    # luminancja mapy docelowej wg współczynników BT.601  
  
distances = np.linspace(dyn_max_dist, dyn_min_dist, iterations).astype(int)  
    # harmonogram Coarse-to-Fine: rozmiar bloku maleje liniowo z każdą iteracją  
  
active_mask = t_luma_block < 240  
    # maska aktywnych pikseli: Luminancja < 240 to obszar twarzy; białe tło jest  
    # wykluczone  
  
s_active = s_flat[mask_flat]  
t_active = t_luma_flat[mask_flat]  
    # wyodrębnienie wyłącznie pikseli twarzy z bloku źródłowego i docelowego  
  
s_active_luma = 0.114*s_active[:, 0] + 0.587*s_active[:, 1] + 0.299*s_active[:, 2]  
    # luminancja aktywnych pikseli bloku źródłowego  
  
s_sort_idx = np.argsort(s_active_luma)  
t_sort_idx = np.argsort(t_active)  
    # indeksy rang jasności: piksele źródłowe i pozycje docelowe posortowane  
    # rosnąco  
  
new_s_active[t_sort_idx] = s_active[s_sort_idx]  
    # transport: k-ty najciemniejszy piksel źródłowy trafia na k-tą pozycję  
    # docelową  
  
new_s_flat[mask_flat] = new_s_active  
    # wpisanie posortowanych pikseli na pozycje aktywne; piksele tła pozostają  
    # niezmienione  
  
S[y:y+curr_dist, x:x+curr_dist] = new_s_flat.reshape(curr_dist, curr_dist, 3)
```



Rys. 16. Wynik sortowania pikseli według luminancji metodą Coarse-to-Fine

11. „Game of Life” Conwaya

Funkcja uruchamia na wyniku detekcji krawędzi Canny'ego nieskończoną symulację automatu komórkowego Conwaya. Implementacja nie korzysta z pętli po pojedynczych pikselach – całe obliczenie następnej generacji realizowane jest przez macierzowy splot 2D z jądrem pierścieniowym 3x3 oraz wektorowe maskowanie tablicy NumPy. Pozwala to na wykonanie pełnej ewolucji siatki w ułamku sekundy niezależnie od rozdzielczości obrazu.

Sąsiedztwo Moorea każdej komórki wyznaczone jest przez splot z jądrem, w którym wartość centralna wynosi 0. Nowa generacja wyliczana jest przez jednoczesne zastosowanie dwóch reguł Conwaya: przetrwania S23 i narodzin B3.

```
cells = (binary_img > 127).astype(np.uint8)
    # mapowanie pikseli do {0, 1}: martwa komórka = 0, żywa = 1

kernel = np.ones((3, 3), dtype=np.uint8)
kernel[1, 1] = 0
    # jądro pierścieniowe 3x3: sumuje 8 sąsiadów Moorea, pomija centrum

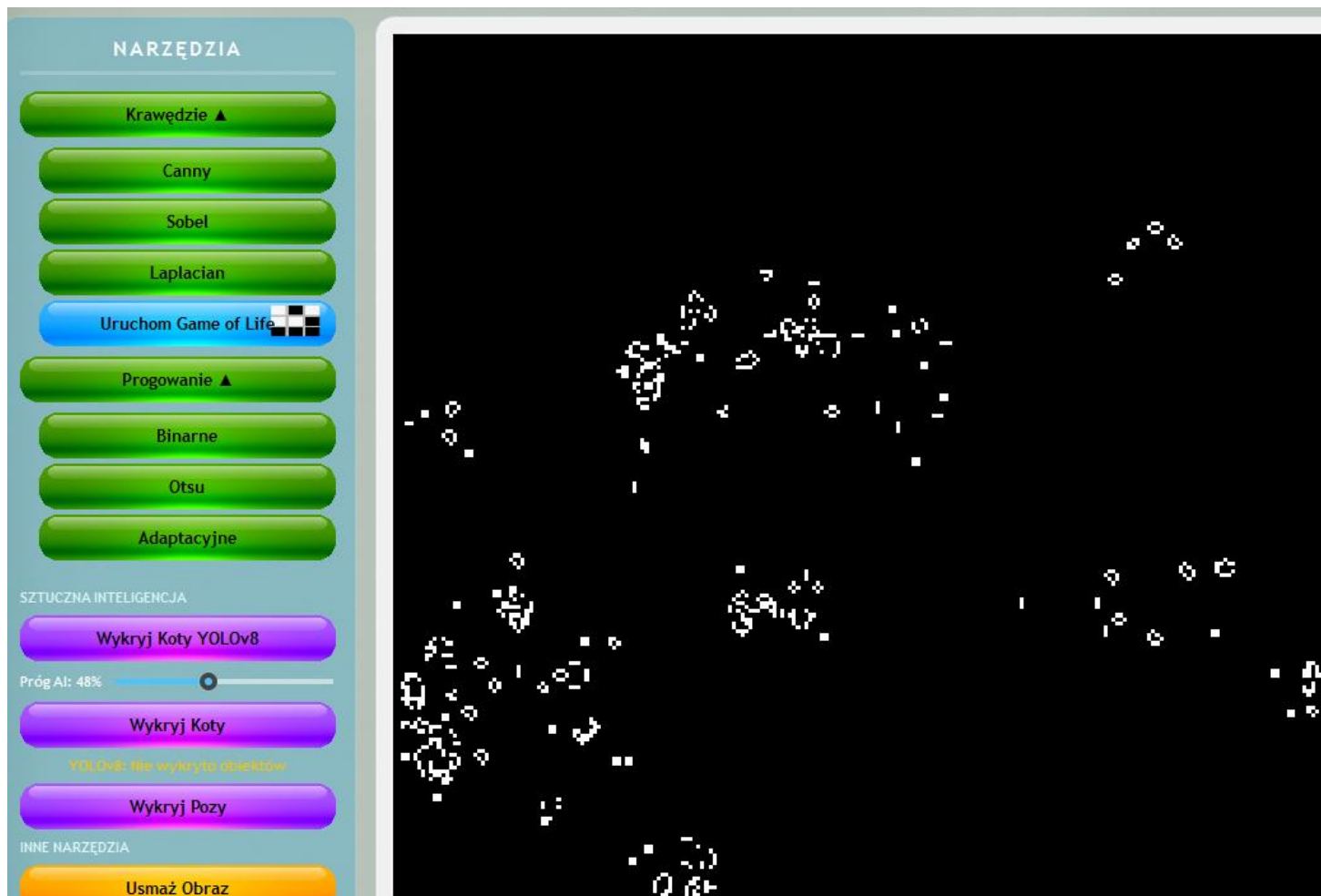
neighbors = cv2.filter2D(cells, -1, kernel,
    borderType=cv2.BORDER_CONSTANT)
    # splot macierzowy: neighbors[i,j] = suma żywych sąsiadów komórki (i,j)
    # operacja wykonywana równolegle dla wszystkich komórek jednocześnie

next_generation = np.zeros_like(cells)

next_generation[(cells == 1) & ((neighbors == 2) | (neighbors == 3))] = 1
    # reguła S23: żywa komórka przeżywa przy 2 lub 3 sąsiadach

next_generation[(cells == 0) & (neighbors == 3)] = 1
    # reguła B3: martwa komórka ożywa przy dokładnie 3 sąsiadach

return (next_generation * 255).astype(np.uint8)
    # konwersja do uint8: żywa komórka = 255, martwa = 0
```

Rys. 17. Automat komórkowy Conwaya uruchomiony na wyniku detekcji krawędzi Canny'ego

Kod źródłowy projektu

Fragmenty kodu zamieszczone przy każdej opisywanej funkcji stanowią skrótową ilustrację matematycznej i algorytmicznej istoty danej metody przetwarzania. Pominęto w nich elementy związane z obsługą interfejsu graficznego, walidacją danych wejściowych i zarządzaniem błędami – pełne implementacje wszystkich opisanych funkcjonalności zawarte są w plikach źródłowych.

- Plik `processor.py` – logika przetwarzania
- Plik `main.py` – interfejs graficzny
- Plik `game_of_life.py` – logika kolejnej generacji gry Game of Life